

Recurrent Neural Networks (RNNs) and LSTMs: Learning from Sequential Data

Modeling sequences, memory, and temporal dependencies in neural networks

by Dr. Rishikesh Yadav

June 04, 2025

Assistant Professor, School of Mathematical and Statistical Sciences, IIT Mandi, India

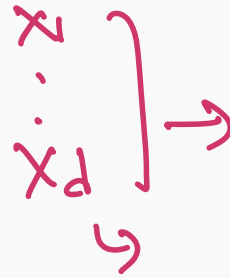
Table of Contents – Motivation

1. Motivation
2. RNNs: Recurrent Neural Networks
 - 2.1 General Introduction
 - 2.2 Vanilla RNNs ✓
 - 2.3 Training of RNNs ✓
 - 2.4 Implementing simple^P RNNs (Vanilla RNNs) in Tensorflow
3. LSTM: Long Short-Term Memory
 - 3.1 Motivation ✓
 - 3.2 LSTM Architecture ✓
 - 3.3 Implementing LSTMs in Tensorflow

Motivation

Motivation

- **Not all problems** can be converted into one with fixed-length inputs and outputs.



Motivation

- **Not all problems** can be converted into one with fixed-length inputs and outputs.
- Some tasks, such as Speech Recognition or Time-series Prediction, require a system to store and use context information.

Motivation

- **Not all problems** can be converted into one with **fixed-length inputs and outputs**.
- Some tasks, such as **Speech Recognition** or **Time-series Prediction**, require a system to **store and use context information**.
- **Example:** Output YES if the number of 1s is even, else NO:

Motivation

- **Not all problems** can be converted into one with **fixed-length inputs and outputs**.
- Some tasks, such as **Speech Recognition** or **Time-series Prediction**, require a system to **store and use context information**.
- **Example:** Output **YES** if the number of 1s is even, else **NO**:
 - 1000010101 → **YES**


Motivation

- **Not all problems** can be converted into one with **fixed-length inputs and outputs**.
- Some tasks, such as **Speech Recognition** or **Time-series Prediction**, require a system to **store and use context information**.
- **Example:** Output **YES** if the number of 1s is even, else **NO**:
 - 1000010101 → **YES**
 - 100011 → **NO**

Motivation

- **Not all problems** can be converted into one with **fixed-length inputs and outputs**.
- Some tasks, such as **Speech Recognition** or **Time-series Prediction**, require a system to **store and use context information**.
- **Example:** Output **YES** if the number of 1s is even, else **NO**:
 - 1000010101 → **YES**
 - 100011 → **NO**
- It is **hard or impossible** to choose a fixed context window:

Motivation

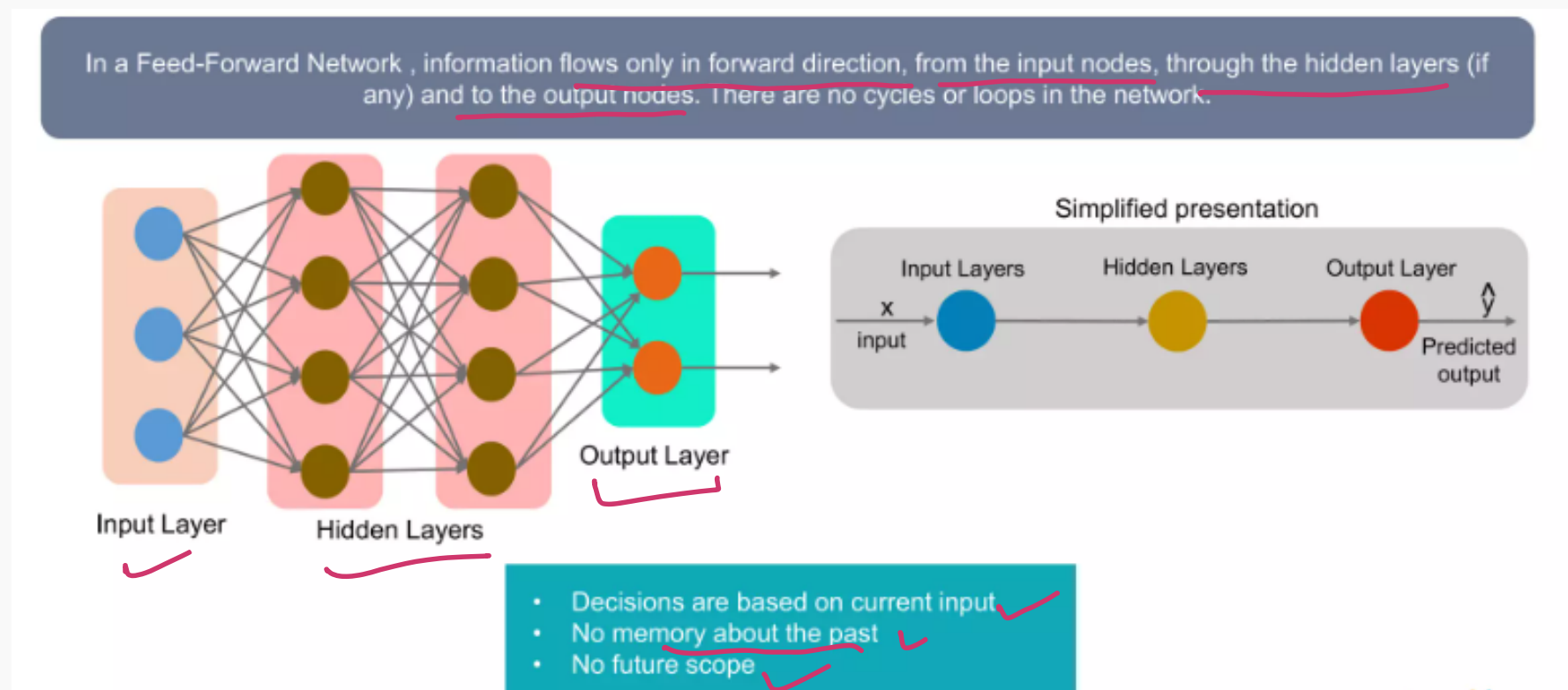
- **Not all problems** can be converted into one with **fixed-length inputs and outputs**.
- Some tasks, such as **Speech Recognition** or **Time-series Prediction**, require a system to **store and use context information**.
- **Example:** Output **YES** if the number of 1s is even, else **NO**:
 - 1000010101 → **YES**
 - 100011 → **NO**
- It is **hard or impossible** to choose a fixed context window:
 - There can always be a **new sample longer** than anything seen before.

Recap: Feedforward Neural Networks (FNNs)

Generally, when we talk about **simple neural networks**, they typically mean **feedforward neural networks (FNNs)** — especially the multi-layer perceptron (MLP) type. These are the foundational architectures used in many early machine learning models.

Recap: Feedforward Neural Networks (FNNs)

Generally, when we talk about **simple neural networks**, they typically mean **feedforward neural networks (FNNs)** — especially the multi-layer perceptron (MLP) type. These are the foundational architectures used in many early machine learning models.



Limitations of FNNs (MLPS)

- ^{MLP} FNNs treat each input as independent — they lack any notion of order or context.

Limitations of FNNs (MLPS)

- FNNs treat each input as independent — they lack any notion of order or context.
- No mechanism to retain information about previous inputs or time steps.

Limitations of FNNs (MLPS)

- FNNs treat each input as independent — they lack any notion of order or context.
- No mechanism to retain information about previous inputs or time steps.
- Fails on tasks where input sequence matters, such as:

Limitations of FNNs (MLPS)

- FNNs treat each input as independent — they lack any notion of order or context.
- No mechanism to retain information about previous inputs or time steps.
- Fails on tasks where input sequence matters, such as:
 - Natural Language: “Dog bites man” $\neq$ “Man bites dog”

Limitations of FNNs (MLPS)

- FNNs treat each input as independent — they lack any notion of order or context.
- No mechanism to retain information about previous inputs or time steps.
- Fails on tasks where input sequence matters, such as:
 - Natural Language: “Dog bites man” $\neq$ “Man bites dog”
 - Time-Series: Predictions require trends from earlier time points

Limitations of FNNs (MLPS)

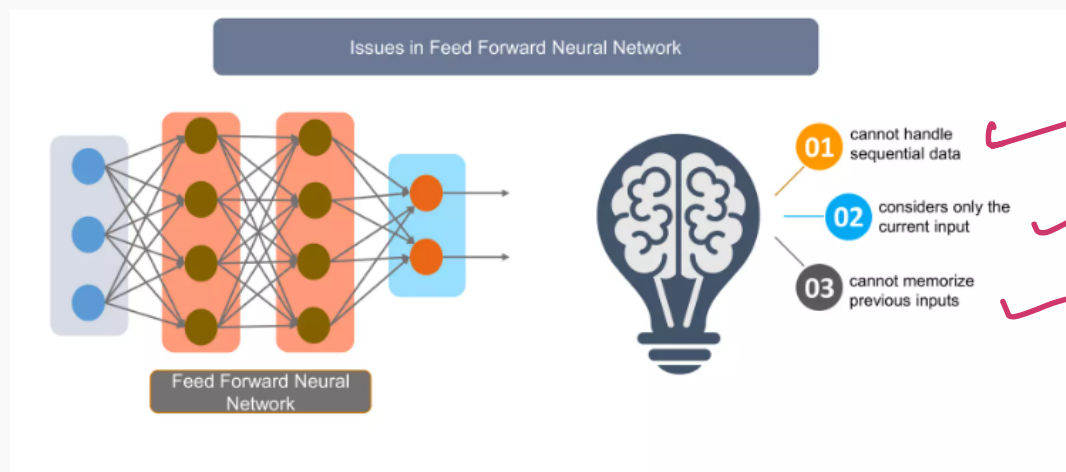
- FNNs treat each input as independent — they lack any notion of order or context.
- No mechanism to retain information about previous inputs or time steps.
- Fails on tasks where input sequence matters, such as:
 - Natural Language: “Dog bites man” $\neq$ “Man bites dog”
 - Time-Series: Predictions require trends from earlier time points
 - Speech/Audio: Context and timing are crucial

Limitations of FNNs (MLPS)

- FNNs treat each input as independent — they lack any notion of order or context.
- No mechanism to retain information about previous inputs or time steps.
- Fails on tasks where input sequence matters, such as:
 - Natural Language: “Dog bites man” $\neq$ “Man bites dog”
 - Time-Series: Predictions require trends from earlier time points
 - Speech/Audio: Context and timing are crucial

Limitations of FNNs (MLPS)

- FNNs treat each input as independent — they lack any notion of order or context.
- No mechanism to retain information about previous inputs or time steps.
- Fails on tasks where input sequence matters, such as:
 - Natural Language: “Dog bites man” $\neq$ “Man bites dog”
 - Time-Series: Predictions require trends from earlier time points
 - Speech/Audio: Context and timing are crucial



FNNs process each input independently: context is lost.

Motivation for RNNs and LSTMs

We need models that can:

- Learn from sequences — where order and context matter
- Retain and reuse past information — like memory
- Adjust their understanding as more input is seen

Motivation for RNNs and LSTMs

We need models that can:

- Learn from sequences — where order and context matter
- Retain and reuse past information — like memory
- Adjust their understanding as more input is seen

Recurrent Neural Networks (RNNs):

- Introduce a feedback loop — current output depends on past hidden states
- Suitable for sequence modeling tasks (language, time-series)
- Struggle with long-term dependencies due to vanishing gradients

$$\begin{aligned} w_{t+1} &= w_t - \eta \frac{\partial}{\partial w_t} \\ w_{t+1} &= w_t \end{aligned}$$

Handwritten notes illustrating the vanishing gradient problem. The first equation shows a weight update rule with a small learning rate η . The second equation shows that the weight remains constant over time. The partial derivative $\frac{\partial}{\partial w_t}$ is shown to approach zero, indicating that the gradient vanishes as it propagates back through time.

Motivation for RNNs and LSTMs

We need models that can:

- Learn from sequences — where order and context matter
- Retain and reuse past information — like memory
- Adjust their understanding as more input is seen

Recurrent Neural Networks (RNNs):

- Introduce a feedback loop — current output depends on past hidden states
- Suitable for sequence modeling tasks (language, time-series)
- Struggle with **long-term dependencies** due to vanishing gradients

Long Short-Term Memory (LSTM):

- Enhances RNNs with **memory cells** and **gating mechanisms**
- Capable of learning **long-range dependencies** effectively
- Avoids vanishing gradient issues via internal memory regulation

RNNs: Recurrent Neural Networks

Agenda

2. RNNs: Recurrent Neural Networks

2.1 General Introduction

2.2 Vanilla RNNs

2.3 Training of RNNs

2.4 Implementing simple RNNs (Vanilla RNNs) in Tensorflow

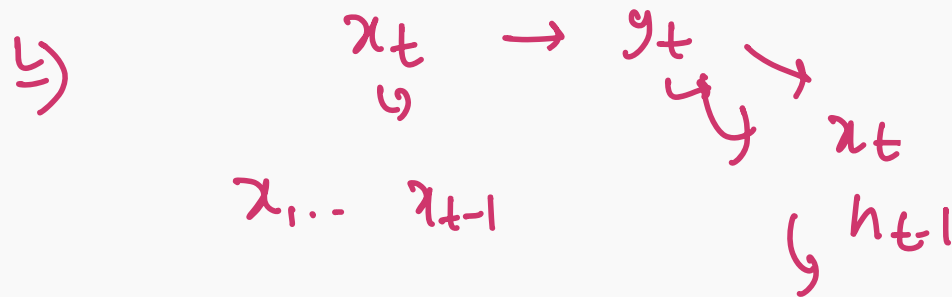
What is a Recurrent Neural Networks (RNNs)?

RNNs are a class of neural networks designed for modeling sequential data. They maintain an internal memory to capture temporal dynamics.

What is a Recurrent Neural Networks (RNNs)?

RNNs are a class of neural networks designed for modeling **sequential data**. They maintain an internal memory to capture temporal dynamics.

- The output at time t depends on the input at t and the hidden state from $t - 1$. That means: RNNs take the **previous output** or **hidden states** as part of the input at each time step.



What is a Recurrent Neural Networks (RNNs)?

RNNs are a class of neural networks designed for modeling sequential data. They maintain an internal memory to capture temporal dynamics.

- The output at time t depends on the input at t and the hidden state from $t - 1$. That means: RNNs take the previous output or hidden states as part of the input at each time step.
- The composite input at time t carries historical information from earlier time steps ($\tau < t$), for example $1, \dots, t - 1$.

What is a Recurrent Neural Networks (RNNs)?

RNNs are a class of neural networks designed for modeling **sequential data**. They maintain an internal memory to capture temporal dynamics.

- The output at time t depends on the input at t and the hidden state from $t - 1$. That means: RNNs take the **previous output** or **hidden states** as part of the input at each time step.
- The composite input at time t carries **historical information** from earlier time steps ($\tau < t$), for example $1, \dots, t - 1$.
- RNNs are powerful because their intermediate values (also called hidden states) can store information about past inputs for a time that is not fixed a priori.

What is a Recurrent Neural Networks (RNNs)?

RNNs are a class of neural networks designed for modeling sequential data. They maintain an internal memory to capture temporal dynamics.

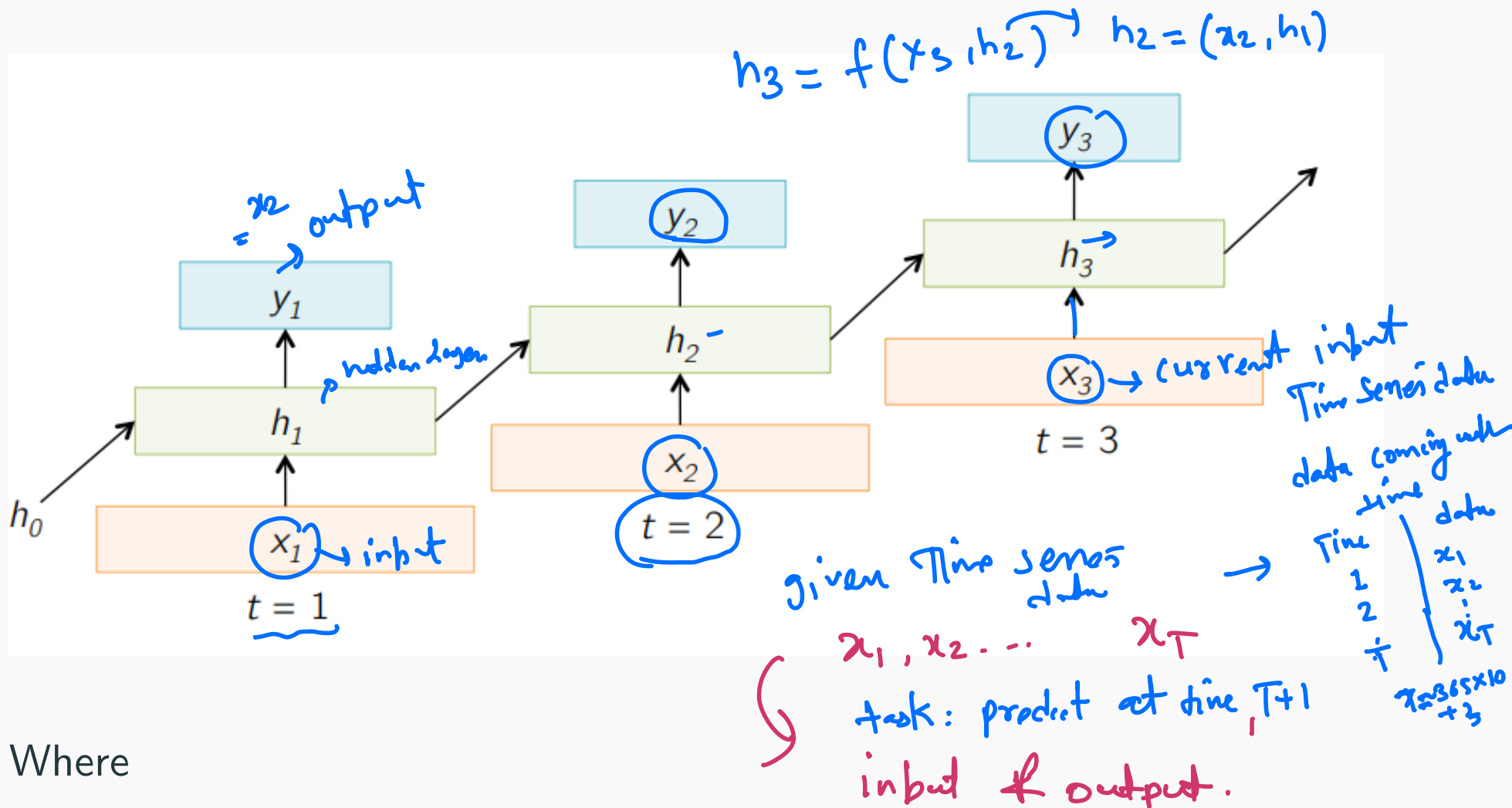
- The output at time t depends on the input at t and the hidden state from $t - 1$. That means: RNNs take the previous output or hidden states as part of the input at each time step.
- The composite input at time t carries historical information from earlier time steps ($\tau < t$), for example $1, \dots, t - 1$.
- RNNs are powerful because their intermediate values (also called hidden states) can store information about past inputs for a time that is not fixed a priori.
- Ideal for problems where context or order matters

What is a Recurrent Neural Networks (RNNs)?

RNNs are a class of neural networks designed for modeling **sequential data**. They maintain an internal memory to capture temporal dynamics.

- The output at time t depends on the input at t and the hidden state from $t - 1$. That means: RNNs take the **previous output** or **hidden states** as part of the input at each time step.
- The composite input at time t carries **historical information** from earlier time steps ($\tau < t$), for example $1, \dots, t - 1$.
- RNNs are powerful because their intermediate values (also called **hidden states**) can **store information** about past inputs for a time that is **not fixed a priori**.
- Ideal for problems where **context or order matters**
- Used in language modeling, speech, and time-series tasks

Sample RNNs



- h_t is the hidden state at time t
- x_t is the input vector at time t

$\Rightarrow$ input x 's

$x_1 \rightarrow$
 $x_2 \rightarrow$
 $\vdots$
 x_{T-1}

outputs y 's

$x_2^* = y_1$
 $x_3^* = y_2$
 $\vdots$
 $x_T^* = y_{T-1}$

Agenda

2. RNNs: Recurrent Neural Networks

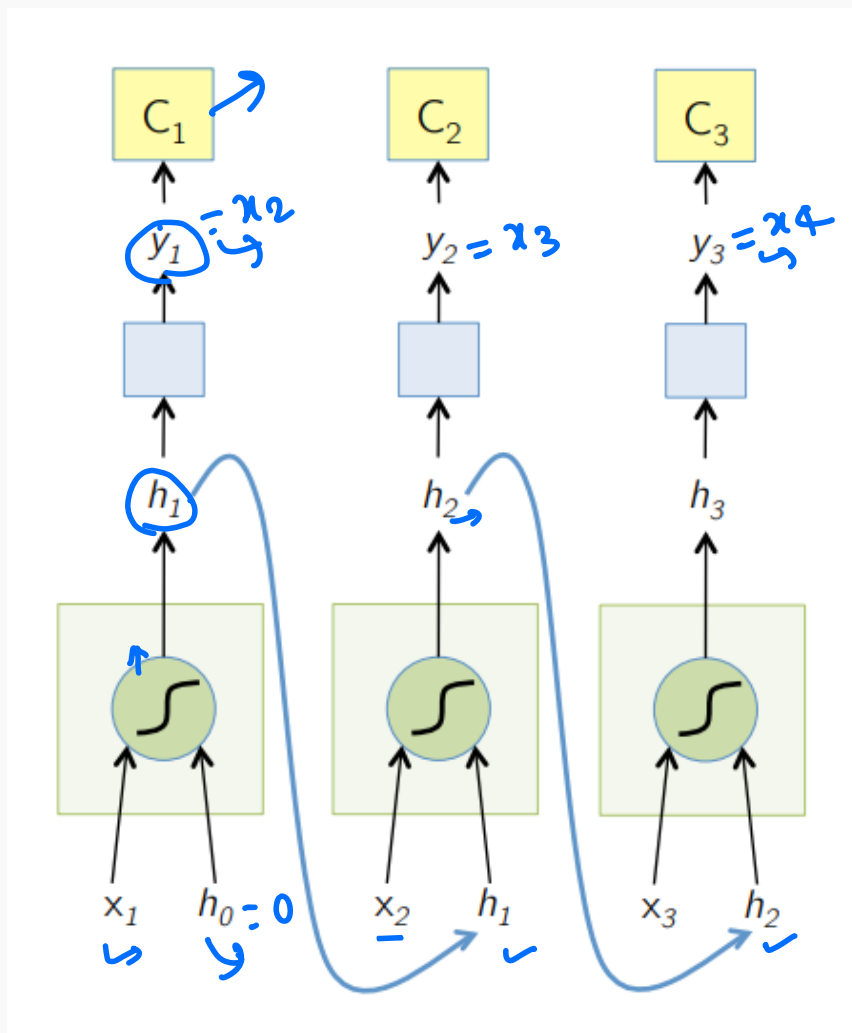
2.1 General Introduction

2.2 Vanilla RNNs

2.3 Training of RNNs

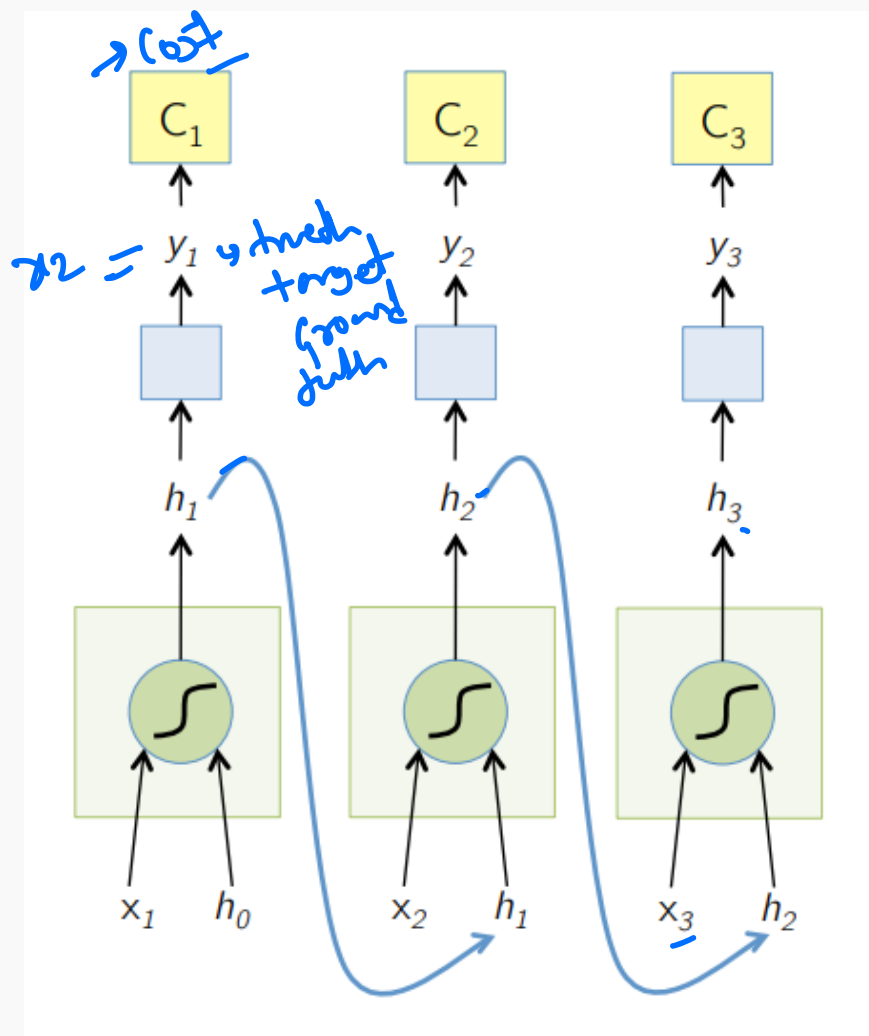
2.4 Implementing simple RNNs (Vanilla RNNs) in Tensorflow

RNNs Forward Steps for (for Vanilla RNNs)



An RNN processes input sequences one step at a time. The same RNN cell (with shared weights) is reused across all time steps. This makes the RNN look like a deep network with depth equal to the sequencelength.

RNNs Forward Steps for (for Vanilla RNNs)



- The basic RNN cell updates as:

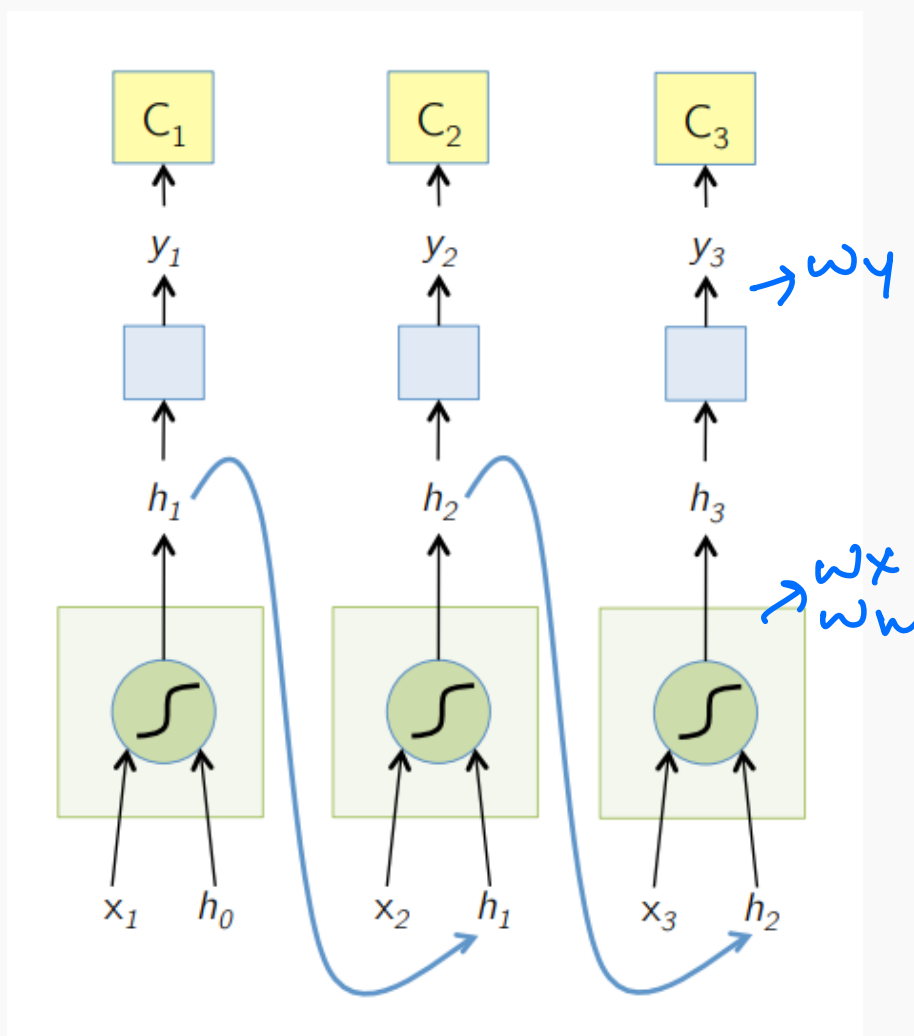
$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$

Handwritten annotations: 'activator fn' with an arrow to the tanh function, 'pre-activation' with an arrow to the sum inside the tanh, and 'post hidden information' with an arrow to the hidden state h_{t-1} .

where W_h , W_x , and b are learnable parameters.

An RNN processes input sequences one step at a time. The same RNN cell (with shared weights) is reused across all time steps. This makes the RNN look like a deep network with depth equal to the sequencelength.

RNNs Forward Steps for (for Vanilla RNNs)



- The basic RNN cell updates as:

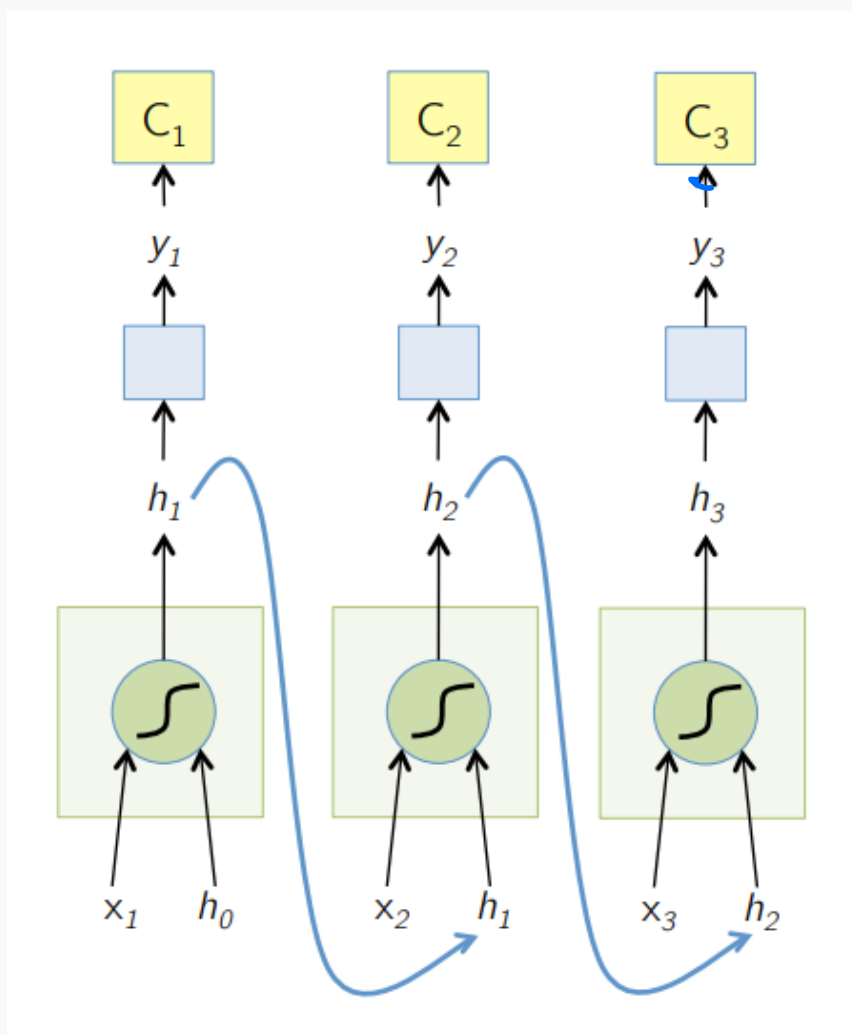
$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$

where W_h , W_x , and b are learnable parameters.

- The output $y_t = \sigma(W_y h_t)$, where $\sigma(\cdot)$ is the activation function for the output layer

An RNN processes input sequences one step at a time. The same RNN cell (with shared weights) is reused across all time steps. This makes the RNN look like a deep network with depth equal to the sequencelength.

RNNs Forward Steps for (for Vanilla RNNs)



An RNN processes input sequences one step at a time. The same RNN cell (with shared weights) is reused across all time steps. This makes the RNN look like a deep network with depth equal to the sequencelength.

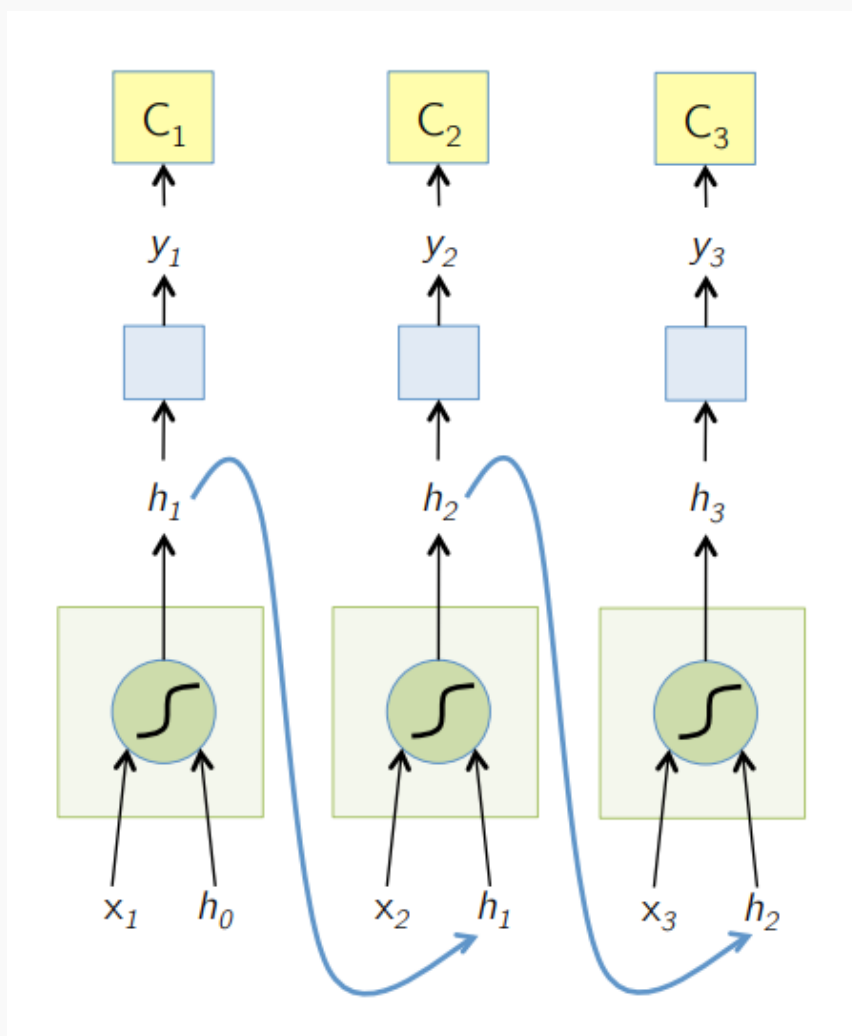
- The basic RNN cell updates as:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$

where W_h , W_x , and b are learnable parameters.

- The output $y_t = \sigma(W_y h_t)$, where $\sigma(\cdot)$ is the activation function for the output layer
- $C_t = \text{Loss}(y_t, \text{GT}_t)$ is the loss calculated at time t

RNNs Forward Steps for (for Vanilla RNNs)



An RNN processes input sequences one step at a time. The same RNN cell (with shared weights) is reused across all time steps. This makes the RNN look like a deep network with depth equal to the sequencelength.

- The basic RNN cell updates as:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$

where W_h , W_x , and b are learnable parameters.

- The output $y_t = \sigma(W_y h_t)$, where $\sigma(\cdot)$ is the activation function for the output layer
- $C_t = \text{Loss}(y_t, \text{GT}_t)$ is the loss calculated at time t
- Note that the weights W_h , W_x , W_y and biases b are shared over time

$\Rightarrow \frac{1}{T} \sum_{t=1}^T C_t = \text{Loss}$

$\angle (W_h, W_x, W_y, b)$

RNNs application: Time-Series Forecasting (e.g., Finance)

- **Goal:** Predict a future financial value using past observations (e.g., next-day price/return).

RNNs application: Time-Series Forecasting (e.g., Finance)

- **Goal:** Predict a future financial value using past observations (e.g., next-day price/return).
- **Inputs (sequence):** A time-ordered window of market features

$$x_{t-L+1}, \dots, x_t$$

, where typically $x_t = [\text{Open}, \text{High}, \text{Low}, \text{Close}, \text{Volume}]$.

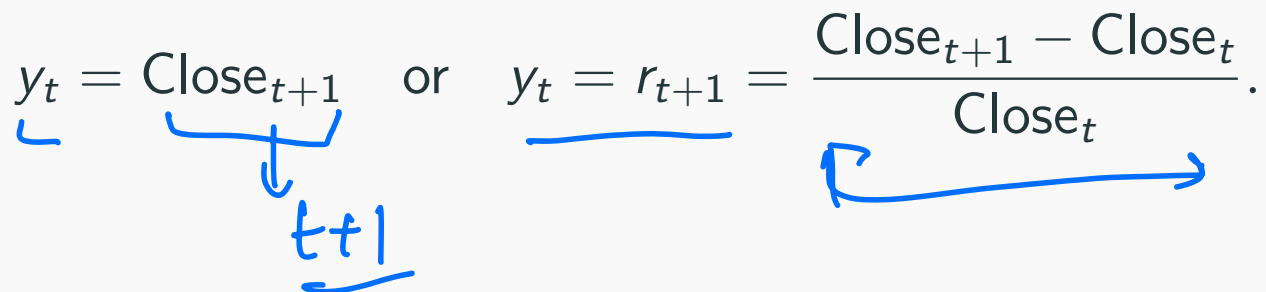
RNNs application: Time-Series Forecasting (e.g., Finance)

- **Goal:** Predict a future financial value using past observations (e.g., next-day price/return).
- **Inputs (sequence):** A time-ordered window of market features

$$x_{t-L+1}, \dots, x_t$$

, where typically $x_t = [\text{Open}, \text{High}, \text{Low}, \text{Close}, \text{Volume}]$.

- **Output (forecast):** One-step ahead target, e.g.,

$$y_t = \text{Close}_{t+1} \quad \text{or} \quad y_t = r_{t+1} = \frac{\text{Close}_{t+1} - \text{Close}_t}{\text{Close}_t}.$$


RNNs application: Time-Series Forecasting (e.g., Finance)

- **Goal:** Predict a future financial value using past observations (e.g., next-day price/return).
- **Inputs (sequence):** A time-ordered window of market features

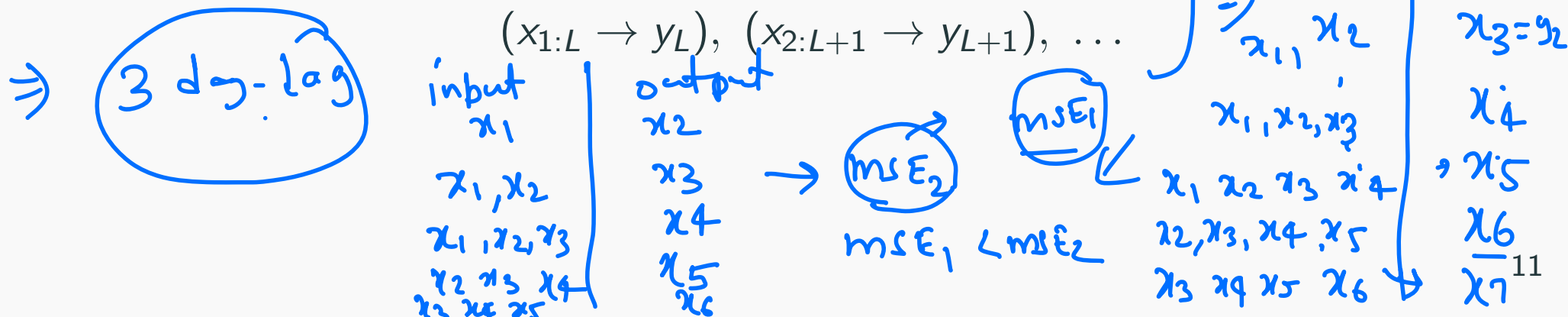
$$X_{t-L+1}, \dots, X_t$$

, where typically $x_t = [\text{Open}, \text{High}, \text{Low}, \text{Close}, \text{Volume}]$.

- **Output (forecast):** One-step ahead target, e.g.,

$$y_t = \text{Close}_{t+1} \quad \text{or} \quad y_t = r_{t+1} = \frac{\text{Close}_{t+1} - \text{Close}_t}{\text{Close}_t} \cdot \text{4 days lag}$$

- **How training pairs are built:** slide the window forward to create many examples:



RNNs application: Time-Series Forecasting (e.g., Finance)

- **Goal:** Predict a future financial value using past observations (e.g., next-day price/return).

- **Inputs (sequence):** A time-ordered window of market features

$$x_{t-L+1}, \dots, x_t$$

, where typically $x_t = [\text{Open}, \text{High}, \text{Low}, \text{Close}, \text{Volume}]$.

- **Output (forecast):** One-step ahead target, e.g.,

$$y_t = \text{Close}_{t+1} \quad \text{or} \quad y_t = r_{t+1} = \frac{\text{Close}_{t+1} - \text{Close}_t}{\text{Close}_t}.$$

- **How training pairs are built:** slide the window forward to create many examples:

$$(x_{1:L} \rightarrow y_L), (x_{2:L+1} \rightarrow y_{L+1}), \dots$$

- **Loss:** regression loss such as MSE: $\mathcal{L} = \frac{1}{N} \sum_i (\hat{y}^{(i)} - y^{(i)})^2$.

RNNs application: Time-Series Forecasting (e.g., Finance)

- **Goal:** Predict a future financial value using past observations (e.g., next-day price/return).

- **Inputs (sequence):** A time-ordered window of market features

$$x_{t-L+1}, \dots, x_t$$

, where typically $x_t = [\text{Open}, \text{High}, \text{Low}, \text{Close}, \text{Volume}]$.

- **Output (forecast):** One-step ahead target, e.g.,

$$y_t = \text{Close}_{t+1} \quad \text{or} \quad y_t = r_{t+1} = \frac{\text{Close}_{t+1} - \text{Close}_t}{\text{Close}_t}.$$

- **How training pairs are built:** slide the window forward to create many examples:

$$(x_{1:L} \rightarrow y_L), (x_{2:L+1} \rightarrow y_{L+1}), \dots$$

- **Loss:** regression loss such as MSE: $\mathcal{L} = \frac{1}{N} \sum_i (\hat{y}^{(i)} - y^{(i)})^2$.
- **Use cases:** forecasting, risk signals, anomaly detection (note: prices are noisy, so evaluate carefully).

RNNs application: Sentiment Classification

- **Goal:** Classify a review (e.g., from [Yelp!](#), [IMDB](#), etc.) as **positive** or **negative**.

RNNs application: Sentiment Classification

- **Goal:** Classify a review (e.g., from [Yelp!](#), [IMDB](#), etc.) as **positive** or **negative**.
- **Inputs:** Multiple words forming one or more sentences.

RNNs application: Sentiment Classification

- **Goal:** Classify a review (e.g., from [Yelp!](#), [IMDB](#), etc.) as **positive** or **negative**.
- **Inputs:** Multiple words forming one or more sentences.
- **Outputs:** Binary classification — Positive / Negative.

RNNs application: Sentiment Classification

- **Goal:** Classify a review (e.g., from [Yelp!](#), [IMDB](#), etc.) as **positive** or **negative**.
- **Inputs:** Multiple words forming one or more sentences.
- **Outputs:** Binary classification — Positive / Negative.
- **Examples:**

RNNs application: Sentiment Classification

- **Goal:** Classify a review (e.g., from **Yelp!**, **IMDB**, etc.) as **positive** or **negative**.
- **Inputs:** Multiple words forming one or more sentences.
- **Outputs:** Binary classification — Positive / Negative.
- **Examples:**
 - "The food was really good." $\Rightarrow$ Positive $y = 1$
input

RNNs application: Sentiment Classification

- **Goal:** Classify a review (e.g., from [Yelp!](#), [IMDB](#), etc.) as **positive** or **negative**.
- **Inputs:** Multiple words forming one or more sentences.
- **Outputs:** Binary classification — Positive / Negative.
- **Examples:**
 - “The food was really good.” ⇒ **Positive**
 - “The chicken crossed the road because it was uncooked.” ⇒ **Negative** ✓

RNNs application: Sentiment Classification

- **Goal:** Classify a review (e.g., from **Yelp!**, **IMDB**, etc.) as **positive** or **negative**.
- **Inputs:** Multiple words forming one or more sentences.
- **Outputs:** Binary classification — Positive / Negative.
- **Examples:**
 - “*The food was really good.*” ⇒ **Positive**
 - “*The chicken crossed the road because it was uncooked.*” ⇒ **Negative**
 - “*Absolutely loved the cinematography and acting.*” ⇒ **Positive**

RNNs application: Sentiment Classification

- **Goal:** Classify a review (e.g., from **Yelp!**, **IMDB**, etc.) as **positive** or **negative**.
- **Inputs:** Multiple words forming one or more sentences.
- **Outputs:** Binary classification — Positive / Negative.
- **Examples:**
 - “The food was really good.” ⇒ **Positive** \
 - “The chicken crossed the road because it was uncooked.” ⇒ **Negative** ▶
 - “Absolutely loved the cinematography and acting.” ⇒ **Positive** \
 - “The plot was dull and completely predictable.” ⇒ **Negative** ◡

RNNs application: Sentiment Classification

- **Goal:** Classify a review (e.g., from **Yelp!**, **IMDB**, etc.) as **positive** or **negative**.
- **Inputs:** Multiple words forming one or more sentences.
- **Outputs:** Binary classification — Positive / Negative.
- **Examples:**
 - *“The food was really good.”* ⇒ **Positive**
 - *“The chicken crossed the road because it was uncooked.”* ⇒ **Negative**
 - *“Absolutely loved the cinematography and acting.”* ⇒ **Positive**
 - *“The plot was dull and completely predictable.”* ⇒ **Negative**
 - *“Would definitely come here again!”* ⇒ **Positive**

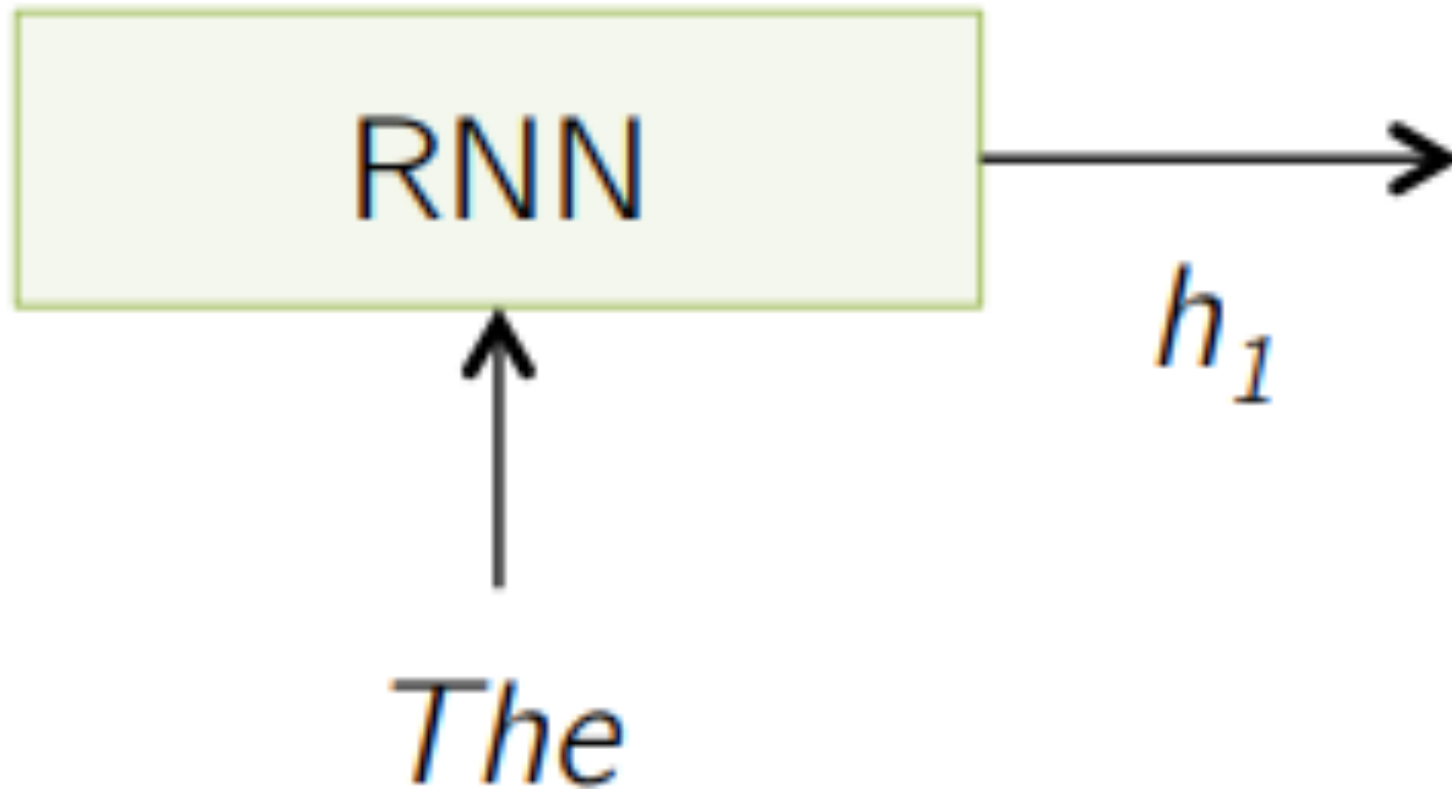
RNNs application: Sentiment Classification

- **Goal:** Classify a review (e.g., from **Yelp!**, **IMDB**, etc.) as **positive** or **negative**.
- **Inputs:** Multiple words forming one or more sentences.
- **Outputs:** Binary classification — Positive / Negative.
- **Examples:**

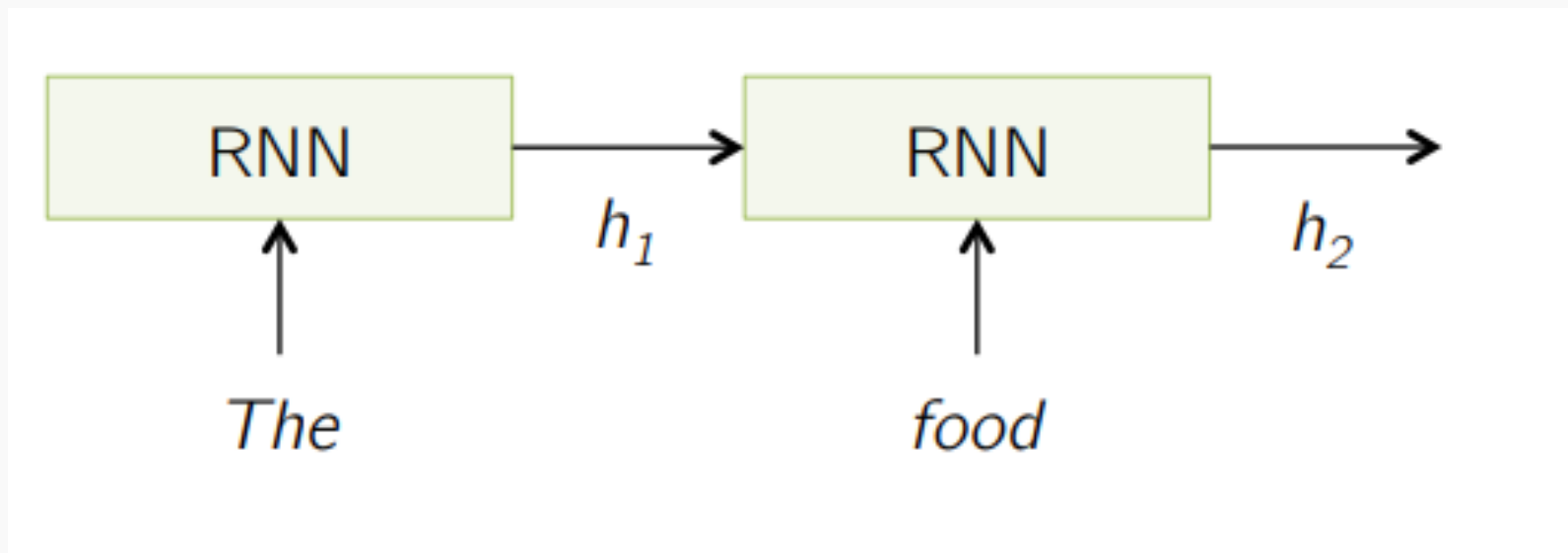
- ✓ Rest) • "The food was really good." ⇒ **Positive** → .
- ✓ Rest • "The chicken crossed the road because it was uncooked." ⇒ **Negative**
- "Absolutely loved the cinematography and acting." ⇒ **Positive**
- "The plot was dull and completely predictable." ⇒ **Negative**
- ✓ Rest • "Would definitely come here again!" ⇒ **Positive**
- ✓ Rest • "Terrible service and cold food." ⇒ **Negative** — 0

Sentiment Classification I

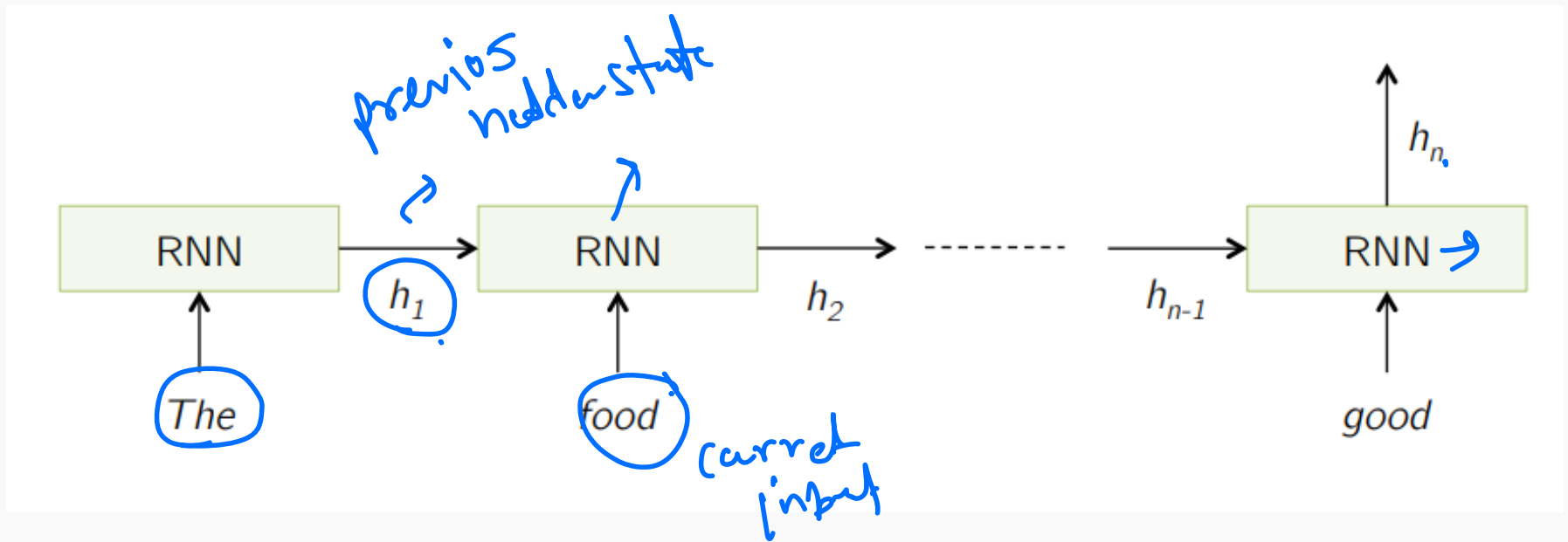
Consider the input: “The food was really good”



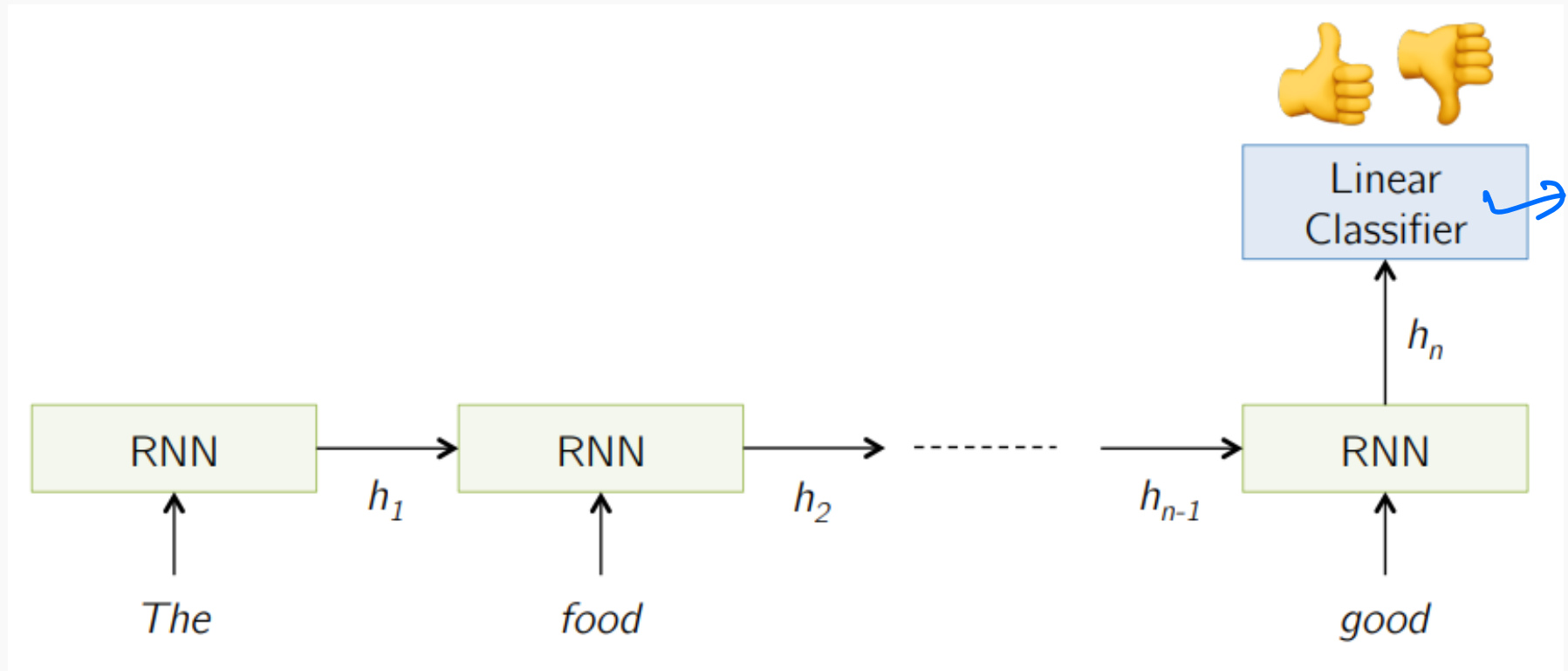
Sentiment Classification II



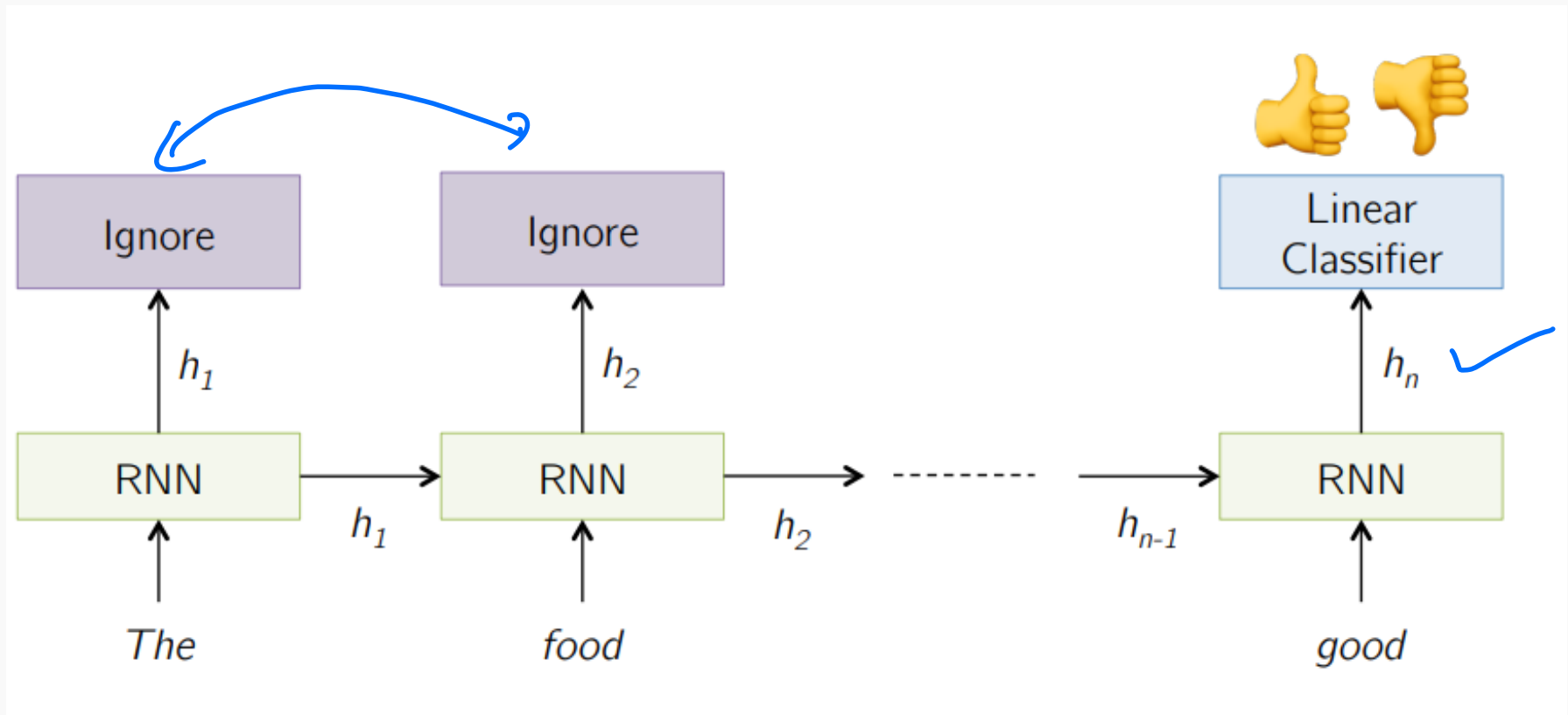
Sentiment Classification III



Sentiment Classification IV



Sentiment Classification V



2. RNNs: Recurrent Neural Networks

2.1 General Introduction

2.2 Vanilla RNNs

2.3 Training of RNNs

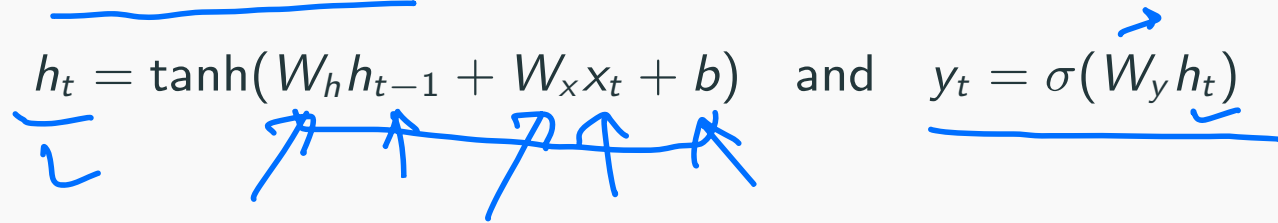
2.4 Implementing simple RNNs (Vanilla RNNs) in Tensorflow

RNNs Backward Steps: Backpropagation Through Time (BPTT)

- Recall the forward RNN steps:

RNNs Backward Steps: Backpropagation Through Time (BPTT)

- Recall the forward RNN steps:
 - Calculation at t -th hidden state

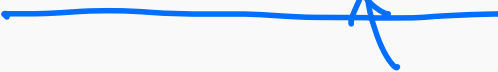
$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b) \quad \text{and} \quad y_t = \sigma(W_y h_t)$$


RNNs Backward Steps: Backpropagation Through Time (BPTT)

- Recall the forward RNN steps:
 - Calculation at t -th hidden state

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b) \quad \text{and} \quad y_t = \sigma(W_y h_t)$$


- The loss at time t :

$$C_t = \text{Loss}(y_t, GT_t)$$


RNNs Backward Steps: Backpropagation Through Time (BPTT)

- Recall the **forward RNN** steps:

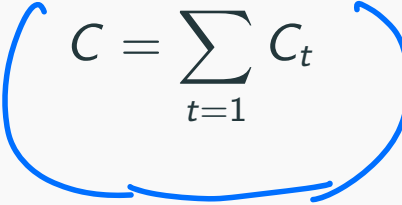
- Calculation at t -th hidden state

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b) \quad \text{and} \quad y_t = \sigma(W_y h_t)$$

- The loss at time t :

$$C_t = \text{Loss}(y_t, \text{GT}_t)$$

- Total loss across T time steps:

$$C = \sum_{t=1}^T C_t$$


RNNs Backward Steps: Backpropagation Through Time (BPTT)

- Recall the forward RNN steps:

- Calculation at t -th hidden state

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b) \quad \text{and} \quad y_t = \sigma(W_y h_t)$$

- The loss at time t :

$$C_t = \text{Loss}(y_t, \text{GT}_t)$$

- Total loss across T time steps:

$$C = \sum_{t=1}^T C_t$$

- The Backpropagation Step:**

RNNs Backward Steps: Backpropagation Through Time (BPTT)

- Recall the **forward RNN** steps:

- Calculation at t -th hidden state

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b) \quad \text{and} \quad y_t = \sigma(W_y h_t)$$

- The loss at time t :

$$C_t = \text{Loss}(y_t, \text{GT}_t)$$

- Total loss across T time steps:

$$C = \sum_{t=1}^T C_t$$

- The Backpropagation Step:**

- Compute gradients $\frac{\partial C}{\partial W_h}$, $\frac{\partial C}{\partial W_x}$, $\frac{\partial C}{\partial W_y}$, $\frac{\partial C}{\partial b}$



RNNs Backward Steps: Backpropagation Through Time (BPTT)

- Recall the **forward RNN** steps:

- Calculation at t -th hidden state

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b) \quad \text{and} \quad y_t = \sigma(W_y h_t)$$

- The loss at time t :

$$C_t = \text{Loss}(y_t, \text{GT}_t)$$

- Total loss across T time steps:

$$C = \sum_{t=1}^T C_t$$

- The Backpropagation Step:**

- Compute gradients $\frac{\partial C}{\partial W_h}$, $\frac{\partial C}{\partial W_x}$, $\frac{\partial C}{\partial W_y}$, $\frac{\partial C}{\partial b}$
- Requires unrolling the network over time and applying the chain rule recursively

RNNs Backward Steps: Backpropagation Through Time (BPTT)

- Recall the **forward RNN** steps:

- Calculation at t -th hidden state

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b) \quad \text{and} \quad y_t = \sigma(W_y h_t)$$

- The loss at time t :

$$C_t = \text{Loss}(y_t, \text{GT}_t)$$

- Total loss across T time steps:

$$C = \sum_{t=1}^T C_t$$

- The Backpropagation Step:**

- Compute gradients $\frac{\partial C}{\partial W_h}$, $\frac{\partial C}{\partial W_x}$, $\frac{\partial C}{\partial W_y}$, $\frac{\partial C}{\partial b}$
- Requires unrolling the network over time and applying the chain rule recursively

- The training is done using well-known optimizers, let's say 'adam'

Agenda

2. RNNs: Recurrent Neural Networks

2.1 General Introduction

2.2 Vanilla RNNs

2.3 Training of RNNs

2.4 Implementing simple RNNs (Vanilla RNNs) in Tensorflow

Implementing RNNs in TensorFlow

We demonstrate how to build a simple RNN using `tf.keras` to perform time series prediction. Open the Google Colab notebook from the Link:

<https://github.com/yadavrishikesh/Masterclass-V2-2026/blob/main/module2/code/RNNs.ipynb>

Step 1: Prepare the Data

- Use a univariate or multivariate time series
- Convert it to input/output sequences using sliding windows

```
1 import numpy as np
2
3 # Example: univariate time series
4 series = np.sin(np.linspace(0, 100, 1000))
5
6 # Prepare X, y sequences
7 def create_sequences(series, window):
8     X, y = [], []
9     for i in range(len(series) - window):
10         X.append(series[i:i+window])
11         y.append(series[i+window])
12     return np.array(X), np.array(y)
13
14 window = 20 → 10
15 X, y = create_sequences(series, window)
16 X = X[..., np.newaxis] # shape: (samples, timesteps,
    features=1)
```

Step 2: Build the RNN Model

- Use SimpleRNN for sequential modeling
- Output a single value for each sequence

```
1 model = tf.keras.Sequential([  
2     tf.keras.layers.SimpleRNN(50, activation='tanh',  
3         input_shape=(X.shape[1], X.shape[2])),  
4     tf.keras.layers.Dense(1)  
5 ])   
6 model.compile(optimizer='adam', loss='mse')  
7 model.summary()
```

Step 3: Train the RNN Model

- Fit the model to predict future values from past sequences
- Use a validation set to monitor performance

```
1 from sklearn.model_selection import train_test_split
2
3 X_train, X_val, y_train, y_val = train_test_split(
4     X, y, test_size=0.2, shuffle=False)
5
6 model.fit(X_train, y_train,
7           validation_data=(X_val, y_val),
8           epochs=20, batch_size=32)
```

Step 4: Make Predictions

- Predict the next value(s) in the sequence

```
1 y_pred = model.predict(X_val)
2
3 # Plot predictions
4 import matplotlib.pyplot as plt
5
6 plt.plot(y_val[:100], label='True')
7 plt.plot(y_pred[:100], label='Predicted')
8 plt.legend(); plt.title("Time Series Prediction")
```

Customization Notes and Summary

- **Customization Notes**

- **Initial states:** Use `initial_state=` argument in `SimpleRNN()` if needed
- **Stacked RNNs:** Set `return_sequences=True` in lower layers
- **Dropout:** `dropout=` and `recurrent_dropout=` to regularize
- **Cell-level API:** Use `tf.keras.layers.RNN(cell)` for full customization
- **Gradients:** TensorFlow automatically handles BPTT

- **Summary**

- ✓ ● RNNs are useful for sequence modeling tasks
- TensorFlow provides high-level RNN APIs (`SimpleRNN`, `LSTM`, `GRU`)
- Training requires careful preprocessing and attention to input shape
- For long sequences, consider `LSTMs` or `GRUs` for better stability

LSTM: Long Short-Term Memory

3. LSTM: Long Short-Term Memory

3.1 Motivation

3.2 LSTM Architecture

3.3 Implementing LSTMs in Tensorflow

Why Do We Need LSTM?

Standard RNNs suffer from:

- Vanishing/exploding gradients: hard to learn long dependencies
- Short-term memory: forgets earlier inputs quickly

Why Do We Need LSTM?

Standard RNNs suffer from:

- Vanishing/exploding gradients: hard to learn long dependencies
- Short-term memory: forgets earlier inputs quickly

LSTM was introduced to overcome the major shortcomings of traditional RNNs. LSTM networks solve this with a unique cell state and gating mechanisms that control information flow.

Why Do We Need LSTM?

Standard RNNs suffer from:

- **Vanishing/exploding gradients**: hard to learn long dependencies
- **Short-term memory**: forgets earlier inputs quickly

LSTM was introduced to overcome the major shortcomings of traditional RNNs. **LSTM networks** solve this with a unique **cell state** and **gating mechanisms** that control information flow.

LSTM can preserve information over long sequences and

- Decide what to remember ✓
- Decide what to forget ✓
- Decide what to output ✓

LSTM: Long Short-Term Memory

LSTM is a special type of Recurrent Neural Network (RNN) designed to learn long-term dependencies. It addresses the limitations of standard RNNs which struggle with vanishing and exploding gradients.

LSTM: Long Short-Term Memory

LSTM is a special type of Recurrent Neural Network (RNN) designed to learn long-term dependencies. It addresses the limitations of standard RNNs which struggle with vanishing and exploding gradients.

- Introduced by Hochreiter and Schmidhuber (1997)

LSTM: Long Short-Term Memory

LSTM is a special type of Recurrent Neural Network (RNN) designed to learn long-term dependencies. It addresses the limitations of standard RNNs which struggle with vanishing and exploding gradients.

- Introduced by Hochreiter and Schmidhuber (1997)
- Effective at learning long-range sequences

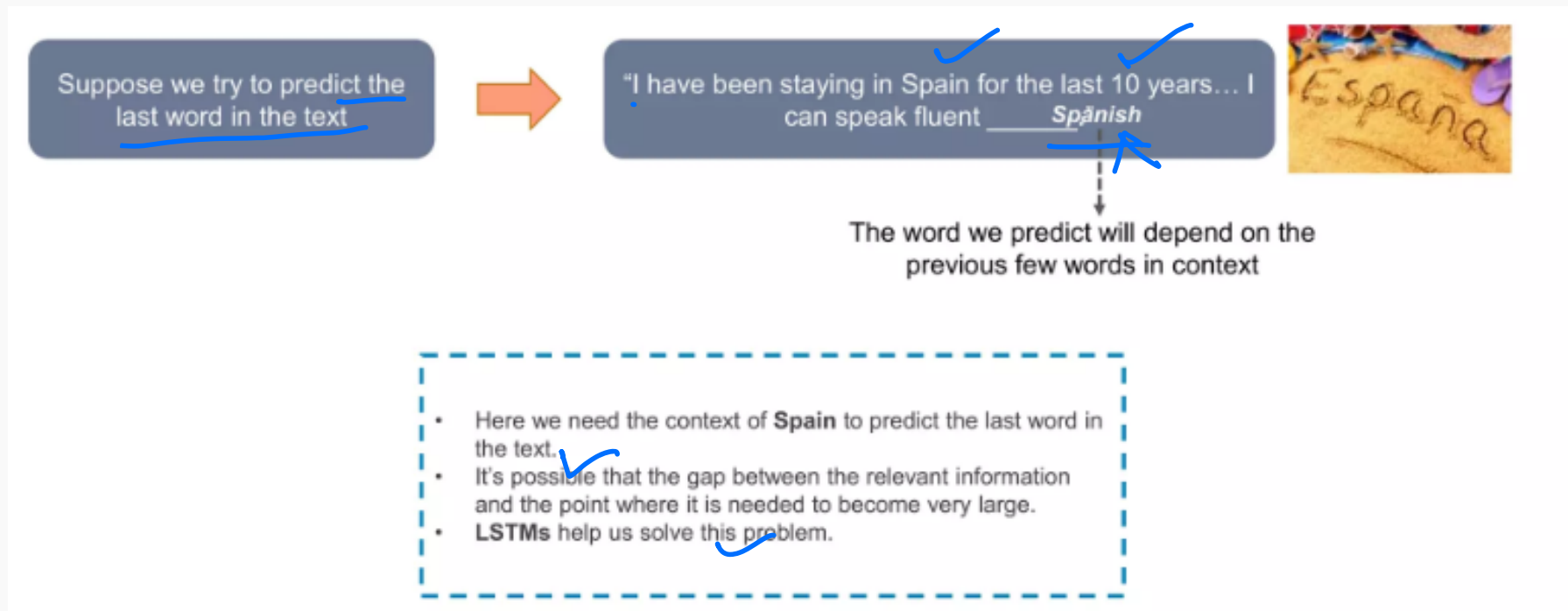


LSTM: Long Short-Term Memory

LSTM is a special type of Recurrent Neural Network (RNN) designed to learn long-term dependencies. It addresses the limitations of standard RNNs which struggle with vanishing and exploding gradients.

- Introduced by Hochreiter and Schmidhuber (1997)
- Effective at learning long-range sequences
- Widely used in NLP, speech recognition, and time series forecasting

Why Long-Term Dependencies is Important?



3. LSTM: Long Short-Term Memory

3.1 Motivation

3.2 LSTM Architecture

3.3 Implementing LSTMs in Tensorflow

LSTM Architecture (1)

- The key feature is the **cell state** (C_t), which runs through the entire chain with only minor interactions.
- Information is added or removed via **gates**:
 - Forget gate f_t ✓
 - Input gate i_t ✓
 - Output gate o_t

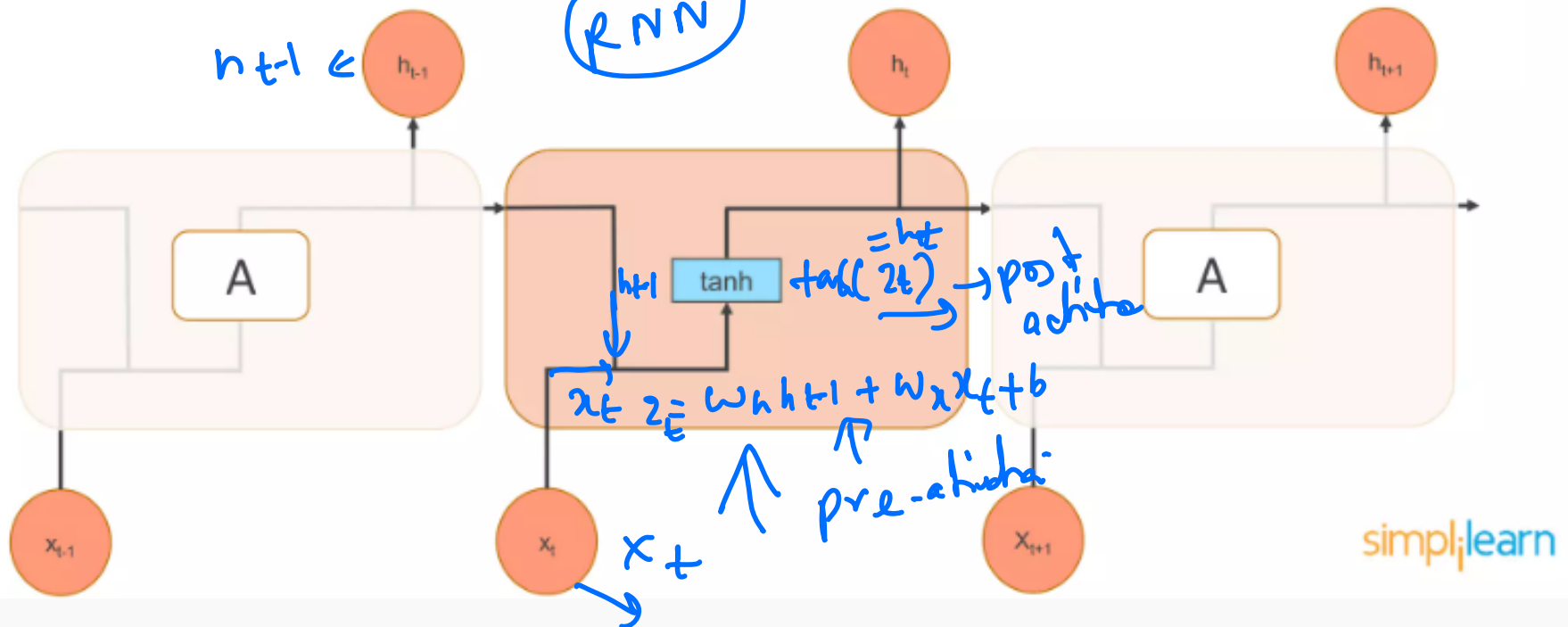
Hidden state h_t carries the output at each time step.



LSTM Architecture (2)

LSTMs are special kind of Recurrent Neural Networks, capable of learning long-term dependencies. Remembering information for long periods of time is their default behavior.

All recurrent neural networks have the form of a chain of repeating modules of neural network. In standard RNNs, this repeating module will have a very simple structure, such as a single tanh layer

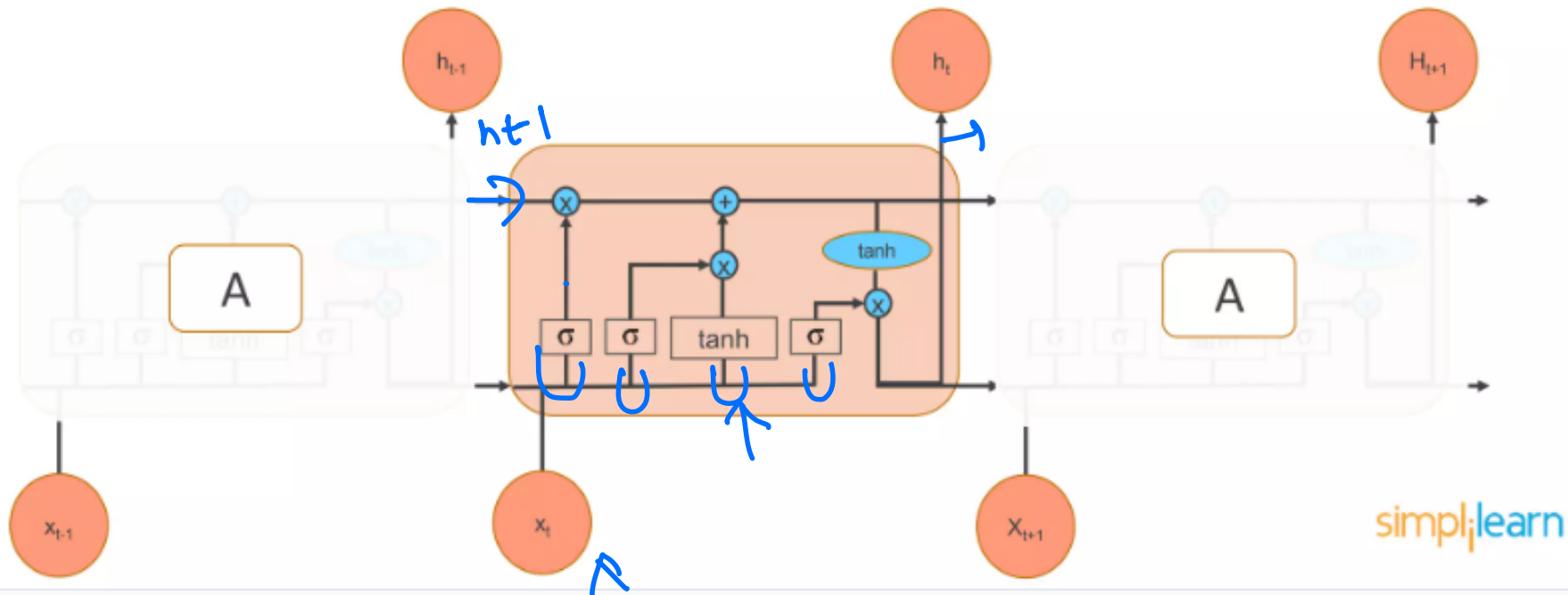


¹<https://www.simplilearn.com/>

LSTM Architecture (3)

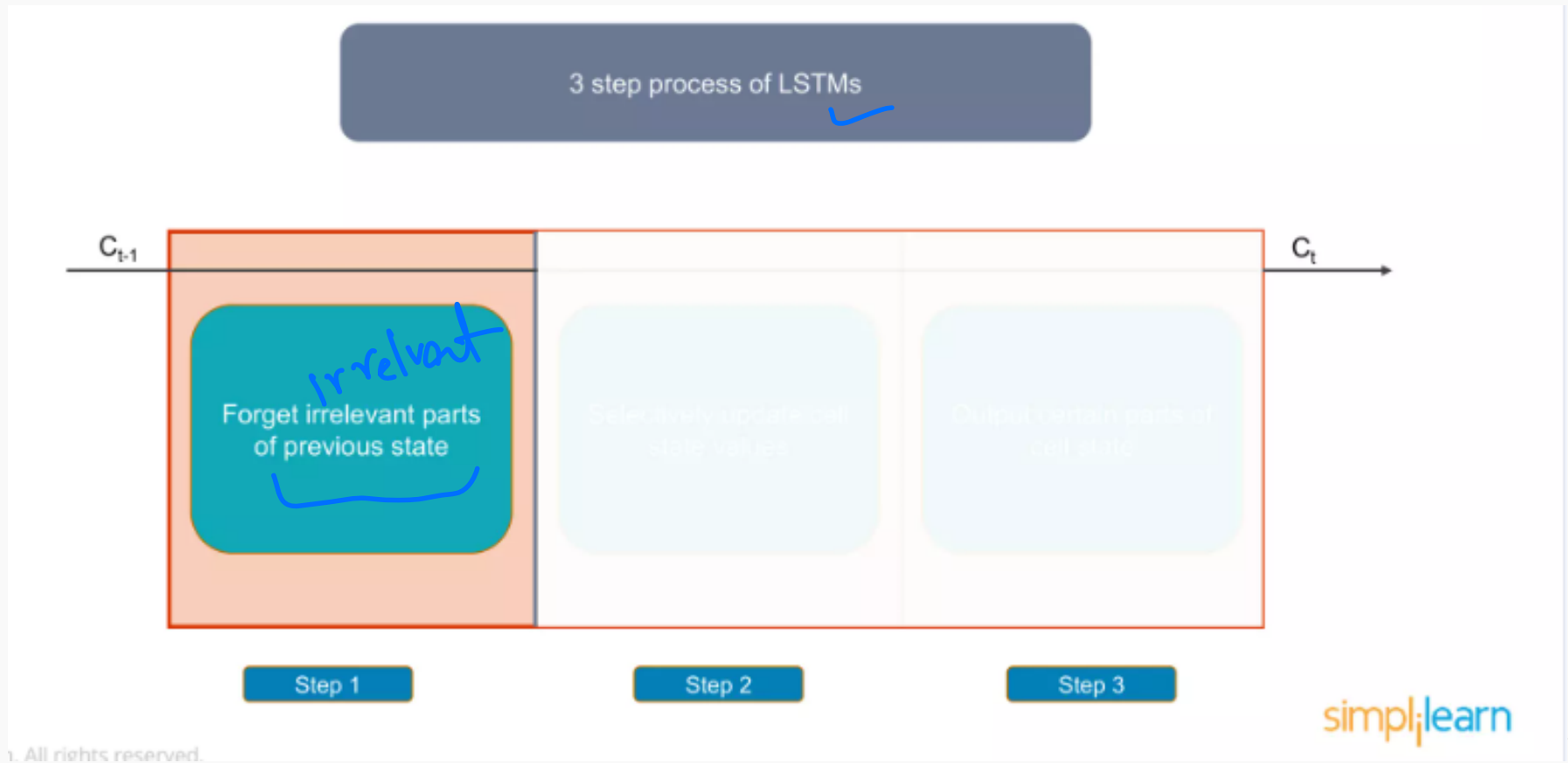
LSTMs are special kind of Recurrent Neural Networks, capable of learning long-term dependencies. Remembering information for long periods of time is their default behavior.

LSTMs also have a chain like structure, but the repeating module has a different structure. Instead of having a single neural network layer, there are four interacting layers communicating in a very special way.



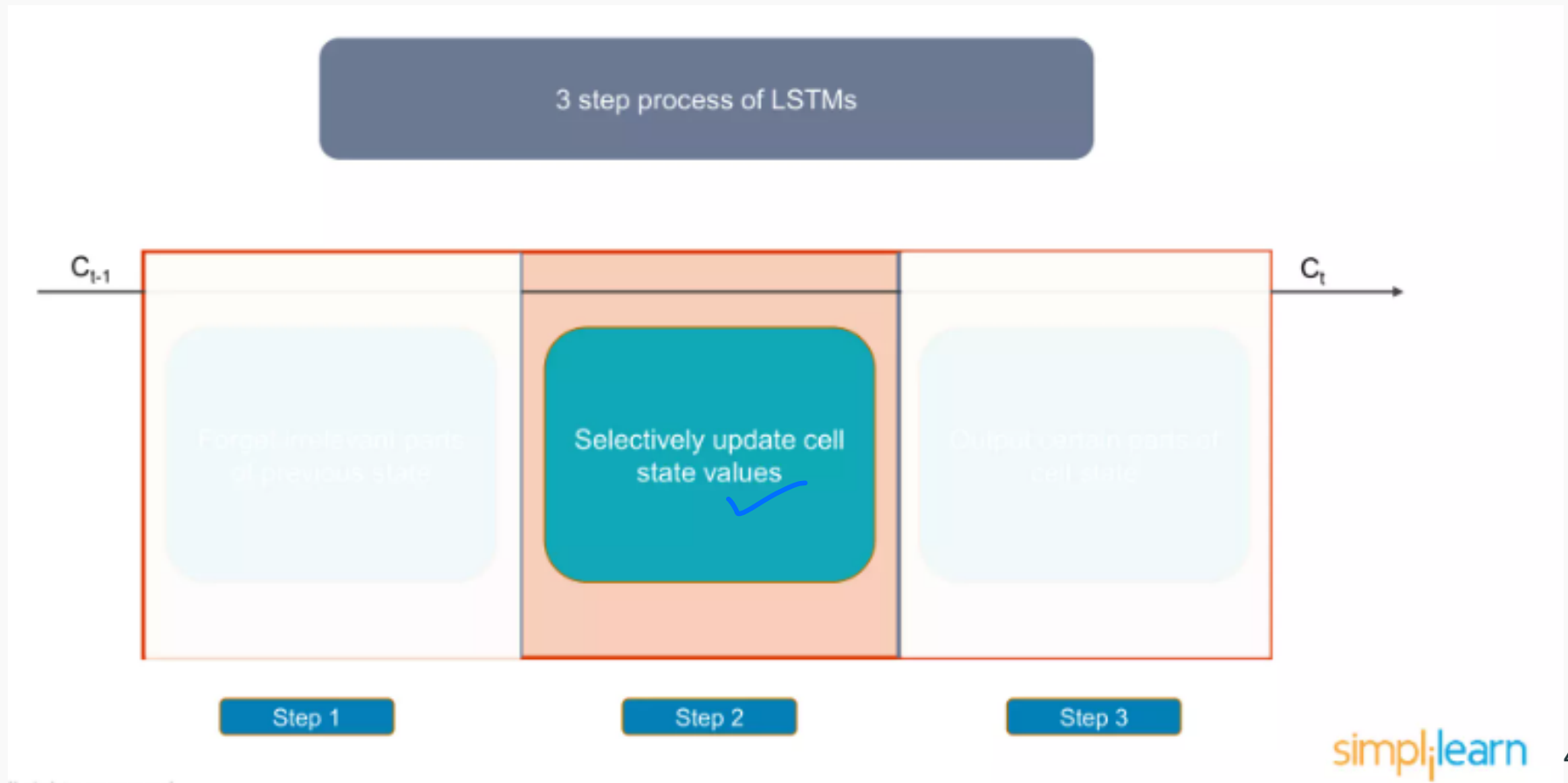
²<https://www.simplilearn.com/>

LSTM Architecture (4)



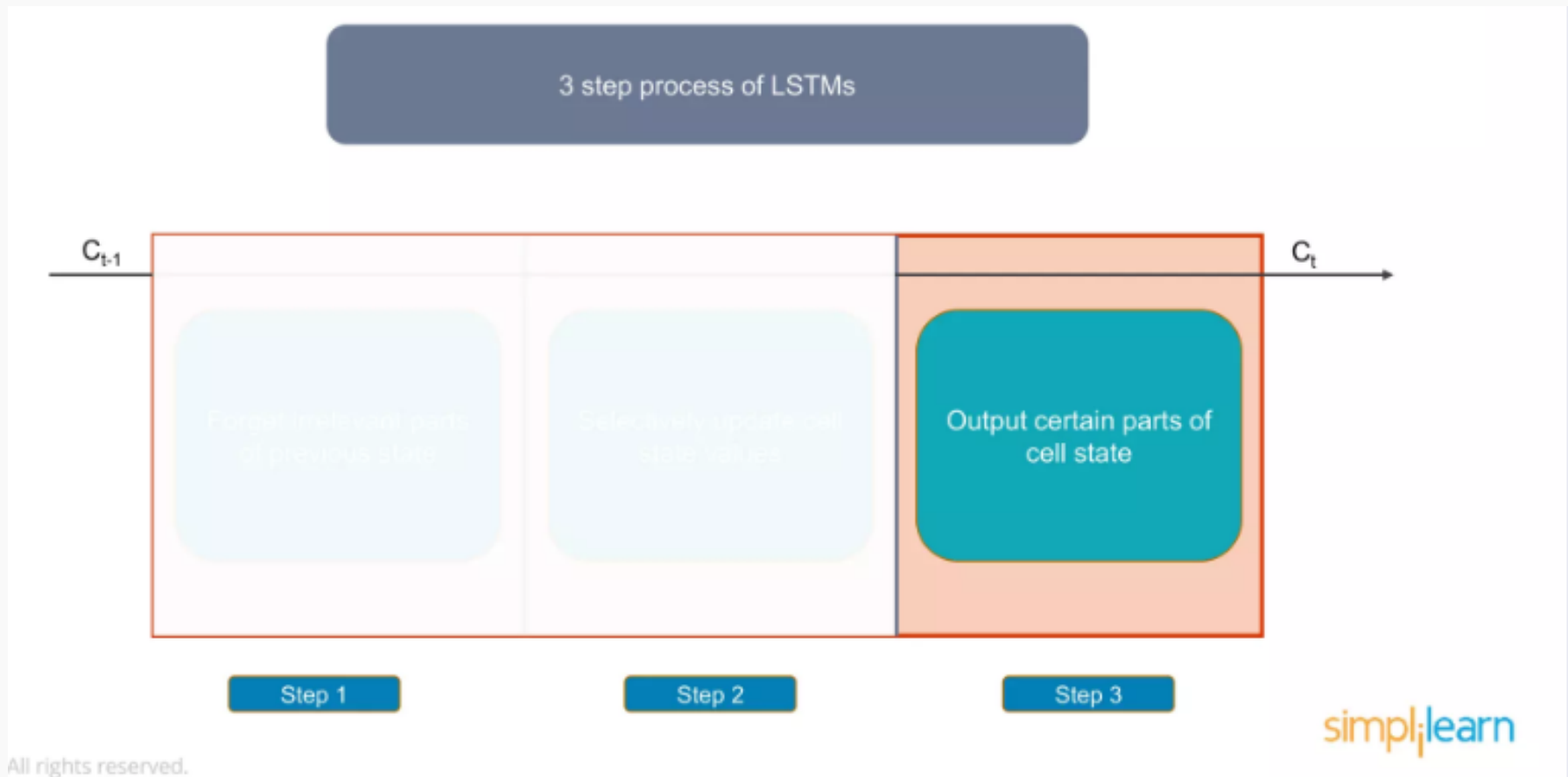
³<https://www.simplilearn.com/>

LSTM Architecture (5)



⁴<https://www.simplilearn.com/>

LSTM Architecture (6)



⁵ <https://www.simplilearn.com/>

Working of LSTMs Cell (1)

Step-1

Decides how much of the past it should remember

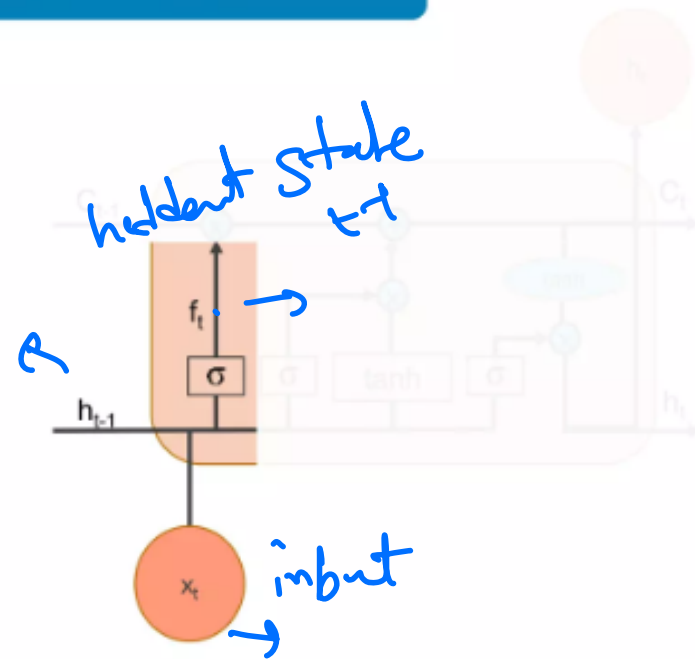
First step in the LSTM is to decide which information to be omitted in from the cell in that particular time step. It is decided by the sigmoid function. It looks at the previous state (h_{t-1}) and the current input x_t and computes the function.

parameters

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

parameters

f_t = forget gate
Decides which information to delete that is not important from previous time step



simplilearn

6

Working of LSTMs Cell (2)

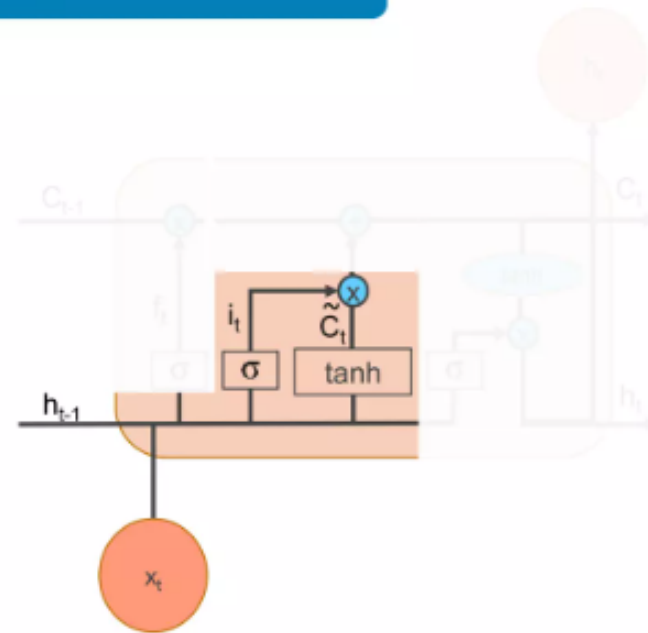
Step-2

Decides how much should this unit add to the current state

In the second layer, there are 2 parts. One is the sigmoid function and the other is the tanh. In the **sigmoid** function, it decides which values to let through (0 or 1). **tanh** function gives the weightage to the values which are passed deciding their level of importance (-1 to 1).

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

i_t = input gate
Determines which information to let through based on its significance in the current time step



simplilearn

7

⁷<https://www.simplilearn.com/>

Working of LSTMs Cell (3)

Step-3

Decides what part of the current cell state makes it to the output

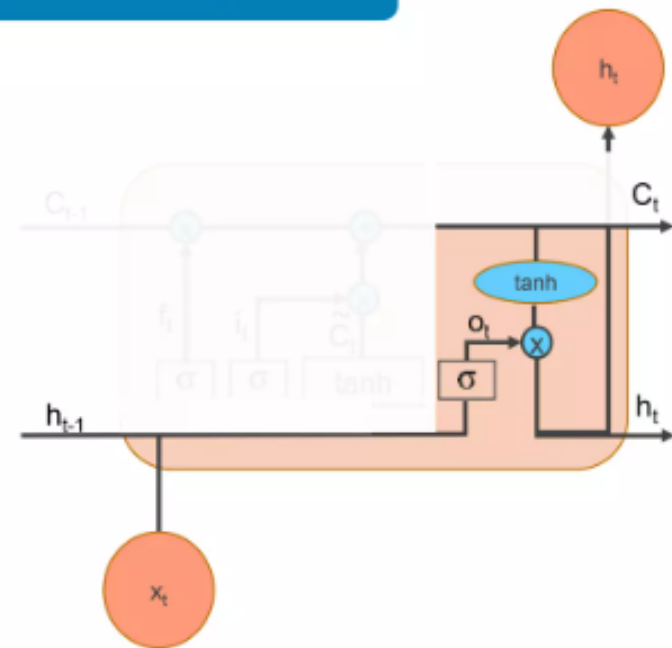
The third step is to decide what will be our output. First, we run a sigmoid layer which decides what parts of the cell state make it to the output. Then, we put the cell state through tanh to push the values to be between -1 and 1 and multiply it by the output of the sigmoid gate.

$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

o_t = output gate

Allows the passed in information to impact the output in the current time step



simplilearn 8

Working of LSTMs Cell (4): LSTM Cell Equations

At time step t , given input x_t , previous hidden state h_{t-1} , and previous cell state C_{t-1} :

h_{t-1}

$$\left\{ \begin{array}{l} f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \\ i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \\ \tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C) \end{array} \right. \quad \begin{array}{l} \text{(Forget Gate)} \\ \text{(Input Gate)} \\ \text{(Candidate Cell State)} \end{array}$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad \text{(Updated Cell State)}$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \quad \text{(Output Gate)}$$

$$h_t = o_t \odot \tanh(C_t) \quad \text{(Hidden State Output)}$$

σ is the sigmoid activation; $\odot$ is element-wise multiplication.

How LSTM Is Trained

- **Forward Pass:**

- Input sequences are passed through the LSTM layer(s).
- At each time step, hidden and cell states are updated using gates.
- The final output is passed to a dense layer to compute predictions.
- A loss (e.g., MSE or cross-entropy) is calculated from predictions.

How LSTM Is Trained

• Forward Pass:

- Input sequences are passed through the LSTM layer(s).
- At each time step, hidden and cell states are updated using gates.
- The final output is passed to a dense layer to compute predictions.
- A loss (e.g., MSE or cross-entropy) is calculated from predictions.

• Backward Pass:

- Gradients of the loss are computed using *Backpropagation Through Time (BPTT)*.
- Gradients flow backward across all time steps and LSTM parameters.
- Optimizer (e.g., Adam) updates weights to minimize the loss.

How LSTM Is Trained

- **Forward Pass:**

- Input sequences are passed through the LSTM layer(s).
- At each time step, hidden and cell states are updated using gates.
- The final output is passed to a dense layer to compute predictions.
- A loss (e.g., MSE or cross-entropy) is calculated from predictions.

- **Backward Pass:**

- Gradients of the loss are computed using *Backpropagation Through Time (BPTT)*.
- Gradients flow backward across all time steps and LSTM parameters.
- Optimizer (e.g., Adam) updates weights to minimize the loss.

- **Repeat for multiple epochs** until validation loss stabilizes or early stopping is triggered.

Advantages of LSTM

- **Handles long sequences:** Able to learn long-term dependencies
- **Gradient flow:** Mitigates vanishing gradient problem
- **Memory mechanism:** Gating allows selective memory and forgetting
- **Versatile:** Works well with sequences of varying length

Compared to traditional RNNs, **LSTMs** are much more stable and effective during training.

Applications of LSTM

LSTM networks are widely used in sequential data problems:

- Natural Language Processing (NLP):

- Text generation ✓
- Machine translation
- Named entity recognition

- Speech Recognition ✓

- Time Series Forecasting ✓

- Video Analysis

↘ CNN + LSTM → Spatio-temporal

LSTM remains a foundational component in many deep learning applications involving temporal data.

3. LSTM: Long Short-Term Memory

3.1 Motivation

3.2 LSTM Architecture

3.3 Implementing LSTMs in Tensorflow

Implementing LSTMs in TensorFlow

We demonstrate how to build a simple LSTM using `tf.keras` to perform time series prediction.

Open the Google Colab notebook from the Link:

<https://github.com/yadavrishikesh/Masterclass-V2-2026/blob/main/module2/code/LSTMs.ipynb>

Preparing Time Series Data for LSTM

- LSTM expects input as 3-D tensors: (samples, timesteps, features).
- Convert raw time series into sequences using a sliding window.

```
1 import numpy as np
2
3 def create_sequences(data, window_size):
4     X, y = [], []
5     for i in range(len(data) - window_size):
6         X.append(data[i:i+window_size])
7         y.append(data[i+window_size])
8     return np.array(X), np.array(y)
9
10 X, y = create_sequences(my_series, window_size=10)
11 X = X[..., np.newaxis]          # Add feature dim for
    univariate
```

LSTM Model Architecture

```
1 from tensorflow.keras.models import Sequential
2 from tensorflow.keras.layers import LSTM, Dense
3
4 model = Sequential([
5     LSTM(64, activation='tanh',
6         input_shape=(X.shape[1], X.shape[2])),
7     Dense(1) # Predict next value
8 ])
9
10 model.compile(optimizer='adam', loss='mse')
11 model.summary()
```

- Stack additional LSTMs or add dropout if needed.
- Adjust the output layer for regression or classification tasks.

Model Training and Evaluation

```
1 history = model.fit(  
2     X_train, y_train,  
3     validation_data=(X_val, y_val),  
4     epochs=20,  
5     batch_size=32,  
6     verbose=1  
7 )  
8  
9 loss = model.evaluate(X_test, y_test) ✓  
10 print(f"Test loss: {loss:.3f}")
```

- Monitor validation loss to avoid over-fitting.
- Consider callbacks like EarlyStopping or ReduceLROnPlateau.

Making Forecasts

```
1 last_sequence = X_test[-1].reshape(1, X.shape[1], X.shape  
   [2])  
2 predicted = model.predict(last_sequence) ✓  
3 print(f"Predicted next value: {predicted[0,0]:.2f}")
```

- Roll forward one step at a time for multi-step forecasts.
- Feed predictions back as input for iterative forecasting.